

The Mathematics of Convolution

From Signals to Matrices

Computer Vision & AI

1. Continuous 1D Convolution

In formal signal processing, the convolution of two continuous functions f and g is defined as the integral of the product of the two functions after one is reversed and shifted.

Integral Definition

$$(f * g)(t) \triangleq \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau$$

- $f(\tau)$: The input signal.
- $g(t - \tau)$: The impulse response (kernel), flipped and shifted by t .
- The integral computes the overlapping area as they slide past each other.

2. Discrete 1D Convolution

In computer science, data is digital (discrete). We replace the continuous integral with a discrete summation over an integer index n .

Summation Definition

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n - m]$$

- For finite arrays of size N and a kernel of size K , the sum becomes finite.
- This operation effectively blends the sequence f using the weights provided by sequence g .

3. Discrete 2D Convolution (For Images)

Images are 2-dimensional discrete spatial signals. We extend the sum to two dimensions (i, j) . Let I be the image and K be the kernel.

2D Convolution Formula

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n)$$

- Note the $i - m$ and $j - n$ indexing. Mathematically, convolution requires **flipping** the kernel K both horizontally and vertically before sliding.

4. Convolution vs. Cross-Correlation

Many Deep Learning libraries (like PyTorch or TensorFlow) actually compute **Cross-Correlation**, not strictly Convolution, though they call it convolution.

Cross-Correlation Formula

$$S(i,j) = (I \star K)(i,j) = \sum_m \sum_n I(i+m, j+n) K(m, n)$$

- **Difference:** No flipping of the kernel ($i+m$ instead of $i-m$).
- **Why?** In Machine Learning, weights are learned. Learning a flipped matrix is the same as learning an unflipped one, so we skip the flip step to save computation.

5. Calculating Output Dimensions

Given an input image of size $W_{in} \times H_{in}$, a kernel of size F , padding P , and stride S , the spatial dimensions of the output feature map are rigorously defined:

Dimension Formula

$$W_{out} = \lfloor \frac{W_{in} - F + 2P}{S} \rfloor + 1$$

$$H_{out} = \lfloor \frac{H_{in} - F + 2P}{S} \rfloor + 1$$

- This formula is critical for designing Neural Network architectures and ensuring tensor shapes match during matrix multiplications.

6. Algebraic Properties

True mathematical convolution exhibits important algebraic properties that make analytical derivations elegant:

- **Commutativity:** $f * g = g * f$
(Filtering an image with a kernel is identical to filtering a kernel with an image).
- **Associativity:** $f * (g * h) = (f * g) * h$
(Applying two filters sequentially is identical to applying their convolution once).
- **Distributivity:** $f * (g + h) = (f * g) + (f * h)$

7. Matrix Multiplication Perspective

Convolution can be expressed purely as matrix multiplication, which is why GPUs process it so quickly.

- The sliding kernel K can be unrolled into a sparse, banded **Toeplitz Matrix**.
- The Image I is flattened into a 1D column vector.
- The entire 2D convolution becomes a single matrix-vector product:
 $\mathbf{y} = \mathbf{T}\mathbf{x}$.

8. Why Convolution for ML?

Why not just use standard dense matrices (Fully Connected Layers)?

- ❶ **Sparse Interactions:** Kernels are much smaller than the image, saving memory ($O(k^2)$ vs $O(N^2)$).
- ❷ **Parameter Sharing:** The same weights (kernel) are used across the whole image, learning universal features like edges.
- ❸ **Translation Equivariance:** $f(g(x)) = g(f(x))$. If the object in the image shifts, the feature map shifts by the exact same amount.